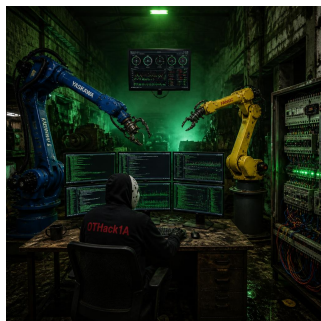




PASSWORD

Guida tecnico-professionale: storia, attacchi e best practice

Creator: 0THack1A

Aggiornamento: Giugno 2026

Documento personale a scopo educativo e divulgativo.
Nessun riferimento aziendale o commerciale.

Ambienti di riferimento per gli esempi tecnici: Debian, Arch Linux, Kali Linux.

Sommario

1	Introduzione	3
1.1	Perché le password restano un problema aperto	3
1.2	Come è organizzato il documento	4
2	Storia ed evoluzione delle password	5
2.1	Le origini: il Compatible Time-Sharing System (1961)	5
2.2	Unix e l'introduzione dell'hashing (anni '70)	5
2.3	Gli anni '90 e 2000: la complessità come (falso) mito	5
2.4	Il punto di svolta: violazioni di massa e credential stuffing (2010-oggi)	6
2.5	Linea del tempo riassuntiva	6
3	Anatomia degli attacchi alle password	8
3.1	Attacco a forza bruta (brute force)	8
3.2	Attacco a dizionario	8
3.3	Credential stuffing	8
3.4	Rainbow table	8
3.5	Phishing e ingegneria sociale	9
3.6	Keylogging e session hijacking	9
3.7	Tabella riassuntiva delle contromisure	9
4	Esempi di attacchi famosi alle password	10
4.1	RockYou (2009): l'archivio che ha cambiato il cracking delle password	10
4.2	Adobe (2013): quando la "crittografia" non è hashing	11
4.3	LinkedIn (2012, rivelato nel 2016): l'hash senza salt	11
4.4	Mirai e l'attacco a Dyn (2016): quando il problema sono le credenziali di fabbrica	11
4.5	Colonial Pipeline (2021): una sola password, nessun MFA	12
4.6	Cosa hanno in comune questi cinque casi	13
5	Crittografia e hashing delle password	14
5.1	Hash vs cifratura: una distinzione fondamentale	14
5.2	L'evoluzione degli algoritmi di hashing	14
5.2.1	MD5 e SHA-1: veloci ma non più adeguati	14
5.2.2	Salting: contro le rainbow table	14
5.2.3	Key stretching: bcrypt, scrypt, Argon2	15
5.3	Peppering: un livello opzionale aggiuntivo	15
5.4	Cosa significa, in pratica, per chi sviluppa software	15
6	Le linee guida NIST e il cambio di paradigma	17
6.1	Dal NIST SP 800-63 (2003) al NIST SP 800-63B (2017+)	17
6.2	Cosa cambia rispetto alle policy tradizionali	17
6.3	Perché la rotazione forzata è stata abbandonata	17
6.4	Perché la complessità obbligatoria non funziona	17
6.5	Il principio guida: lunghezza > complessità	18
6.6	Cosa NIST raccomanda concretamente (sintesi)	18

7	Password manager	19
7.1	Il problema che risolvono	19
7.2	Come funzionano: il modello di sicurezza	19
7.3	Tipologie di password manager	20
7.4	La master password: l'unico punto debole rimasto	20
7.5	Obiezioni comuni e risposta tecnica	21
7.6	Criteri pratici per scegliere un password manager	21
8	Autenticazione a più fattori (MFA/2FA)	22
8.1	Il principio: qualcosa che sai, hai, sei	22
8.2	Tipologie di secondo fattore, dalla meno alla più sicura	22
8.2.1	SMS OTP (codice via SMS)	22
8.2.2	App di autenticazione TOTP (Time-based One-Time Password)	22
8.2.3	Notifiche push	22
8.2.4	Chiavi di sicurezza hardware (FIDO2/U2F)	23
8.3	Tabella comparativa	23
8.4	Raccomandazione pratica	23
9	Passwordless e passkey	24
9.1	Perché eliminare del tutto la password	24
9.2	FIDO2 e WebAuthn: le basi tecniche	24
9.3	Le passkey: WebAuthn reso utilizzabile da tutti	24
9.4	Perché le passkey sono resistenti al phishing	24
9.5	Limiti e considerazioni pratiche attuali	25
9.6	Stato di transizione: convivenza con le password	25
10	Errori comuni da evitare	26
10.1	Lato utente	26
10.2	Lato sviluppatore / amministratore di sistema	26
10.3	Errori di percezione comuni	27
11	Checklist operativa di sintesi	28
11.1	Per l'utente	28
11.2	Per chi sviluppa o amministra sistemi	28
	Glossario	30
	Cheat Sheet	32

CAPITOLO 1

Introduzione

La password è, ancora oggi, il meccanismo di autenticazione più diffuso al mondo. Nonostante decenni di critiche, vulnerabilità note e alternative tecnicamente superiori, continua a essere il primo (e spesso unico) livello di protezione di account email, home banking, infrastrutture aziendali, sistemi industriali e dispositivi personali.

Questo documento nasce con un obiettivo preciso: fornire una guida tecnico-professionale, ma non aziendale, che metta insieme tre piani di lettura complementari:

- **Storico:** come è cambiato il concetto di password dagli albori di Internet a oggi, e perché certe regole "tradizionali" sono oggi considerate superate.
- **Tecnico:** come funzionano realmente gli attacchi alle password e come vengono memorizzate in modo sicuro (hashing, salting, key stretching).
- **Pratico:** quali sono le best practice aggiornate (NIST, password manager, MFA, passkey) e un cheat sheet finale utilizzabile da chiunque, anche senza background di sicurezza.

INFO

Il documento è pensato per un pubblico tecnico (sviluppatori, amministratori di sistema, ingegneri, appassionati di cybersecurity) ma resta leggibile anche da un utente avanzato non specialista. Non contiene riferimenti a politiche aziendali specifiche: le indicazioni sono generali e applicabili a qualsiasi contesto, personale o professionale.

1.1 Perché le password restano un problema aperto

Il problema delle password non è (solo) tecnologico: è anche cognitivo e comportamentale. Un essere umano deve:

1. Creare credenziali diverse per decine, a volte centinaia, di servizi.
2. Ricordarle, o trovare un modo sicuro per non doverlo fare.
3. Aggiornarle quando un servizio subisce una violazione.
4. Resistere a tecniche di ingegneria sociale e phishing sempre più sofisticate.

Il risultato, misurato in modo consistente da analisi su credenziali compromesse pubblicate negli anni, è che la maggioranza degli utenti riutilizza le stesse password su più servizi, sceglie pattern predicibili (nomi, date, sequenze di tastiera) e modifica le credenziali solo dopo un incidente, quasi mai prima.

1.2 Come è organizzato il documento

Capitolo	Contenuto
2	Evoluzione storica: da Unix e Multics agli standard moderni
3	Anatomia degli attacchi: brute force, dizionario, credential stuffing, phishing
4	Esempi di attacchi famosi alle password: RockYou, Adobe, LinkedIn, Mirai, Colonial Pipeline
5	Crittografia e hashing: da MD5 a Argon2
6	Le linee guida NIST SP 800-63B e il cambio di paradigma
7	Password manager: funzionamento, modelli di minaccia, scelta
8	Autenticazione a più fattori (MFA/2FA)
9	Passwordless e passkey (FIDO2 / WebAuthn)
10	Errori comuni da evitare, lato utente e lato sviluppatore
11	Checklist operativa di sintesi

Seguono un **glossario** dei termini tecnici utilizzati e un **cheat sheet** finale di una pagina, pensato per essere consultato rapidamente o stampato.

Fonti

- NIST, *SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management* — <https://pages.nist.gov/800-63-3/sp800-63b.html>
- OWASP Cheat Sheet Series, *Password Storage Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Cheat Sheet Series, *Authentication Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

CAPITOLO 2

Storia ed evoluzione delle password

2.1 Le origini: il Compatible Time-Sharing System (1961)

Il primo sistema informatico documentato a utilizzare password per limitare l'accesso a singoli utenti è il **CTSS** (Compatible Time-Sharing System) del MIT, introdotto nei primi anni '60. In un sistema time-sharing, più utenti condividevano lo stesso computer centrale: la password serviva semplicemente a separare i file e il tempo di calcolo di un utente da quelli di un altro, non a difendersi da un avversario esterno via rete (che, semplicemente, non esisteva ancora).

INFO

È curioso notare che il primo "furto di password" documentato avvenne proprio su CTSS, secondo il racconto reso pubblico dallo stesso protagonista, Allan Scherr, solo decenni dopo i fatti: un ricercatore stampò il file delle password in chiaro per ottenere più tempo macchina di quello allocato al suo account. Le fonti secondarie riportano date leggermente diverse per l'episodio (tra il 1962 e il 1966): la cronologia esatta resta incerta, ma il fatto e la lezione che ne deriva sono ben documentati. La lezione, già allora, era chiara: conservare le password in chiaro è un rischio.

2.2 Unix e l'introduzione dell'hashing (anni '70)

Con Unix, Robert Morris Sr. e Ken Thompson introdussero un'innovazione fondamentale: le password non venivano più salvate in chiaro, ma come risultato di una funzione di hashing unidirezionale (basata sul cifrario DES) memorizzata nel file `/etc/passwd`. Il principio cardine, valido ancora oggi, è semplice:

BEST PRACTICE

Un sistema non dovrebbe mai avere bisogno di conoscere la password in chiaro per verificarla: è sufficiente confrontare l'hash della password inserita con l'hash memorizzato.

Il file `/etc/passwd` era però leggibile da tutti gli utenti del sistema, e con l'aumento della potenza di calcolo divenne possibile eseguire attacchi a dizionario offline contro gli hash. La risposta, alla fine degli anni '80, fu lo **shadow password system** (`/etc/shadow`), che spostò gli hash in un file leggibile solo da root.

2.3 Gli anni '90 e 2000: la complessità come (falso) mito

Con la diffusione del web e dei servizi online, il numero di account per utente crebbe rapidamente. Le organizzazioni risposero imponendo policy sempre più rigide:

- Lunghezza minima (es. 8 caratteri)
- Obbligo di maiuscole, minuscole, numeri e simboli
- Scadenza forzata ogni 30/60/90 giorni

- Divieto di riutilizzo delle ultime N password

Queste regole, formalizzate anche nel celebre documento NIST SP 800-63 del 2003 redatto da Bill Burr, sono rimaste lo standard de facto per quasi 15 anni. Il problema è che si sono rivelate, con il tempo, controproducenti.

ATTENZIONE

Bill Burr, autore delle linee guida del 2003, ha pubblicamente dichiarato anni dopo di "essersi pentito" di molte delle raccomandazioni date. La ricerca successiva ha dimostrato che la rotazione forzata e i requisiti di complessità spingono gli utenti verso pattern predicibili (es. `Password1!` → `Password2!`) piuttosto che verso password realmente più sicure.

2.4 Il punto di svolta: violazioni di massa e credential stuffing (2010-oggi)

A partire dagli anni 2010, una serie di violazioni di dati su larga scala (centinaia di milioni di credenziali esposte) ha cambiato radicalmente la percezione del problema. Non si trattava più solo di "indovinare" una password, ma di riutilizzare combinazioni email/password già compromesse su altri servizi: il **credential stuffing**, descritto nel capitolo 3.

Questo ha portato a due cambiamenti di paradigma fondamentali, ancora in corso:

1. Il passaggio da "password complesse e brevi" a "password lunghe e memorizzabili" (passphrase), sancito dalle linee guida NIST SP 800-63B (2017, aggiornate negli anni successivi).
2. Lo spostamento del problema dall'utente al sistema: autenticazione a più fattori, password manager, e infine autenticazione **senza password** tramite passkey basate su crittografia a chiave pubblica (FIDO2/WebAuthn).

2.5 Linea del tempo riassuntiva

Periodo	Evento principale
1961	CTSS (MIT): prima password di sistema, per separare utenti, non per sicurezza
1970s	Unix: hashing delle password, file <code>/etc/passwd</code>
1987-1990	Diffusione del sistema shadow , separazione hash/permessi
2003	NIST SP 800-63 (Burr): nascita delle policy di complessità classiche
2009-2013	Adozione diffusa di bcrypt/scrypt al posto di MD5/SHA1
2013	Violazione Adobe (150M credenziali): caso di studio su crittografia debole
2017	NIST SP 800-63B: abbandono della rotazione forzata e della complessità obbligatoria
2019-oggi	Diffusione di FIDO2/WebAuthn e passkey come autenticazione passwordless

Fonti

- J. Yan/Wired (rip. Gizmodo), *The World's First Computer Password? It Was Useless Too* — <https://gizmodo.com/the-world-s-first-computer-password-it-was-useless-too-5879856>
- D. Kalat, *The Case of the Purloined Password*, ThinkSet/BRG — <https://www.thinkbrg.com/thinkset/ts-case-of-purloined-password/>
- R. McMillan, *The Man Who Wrote Those Password Rules Has a New Tip*, The Wall Street Journal (ago. 2017) — riportato in BizTech Magazine, <https://biztechmagazine.com/article/2017/0>

8/password-rule-guru-bill-burr-now-regrets-his-advice

- NIST, *SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management* — <https://pages.nist.gov/800-63-3/sp800-63b.html>

CAPITOLO 3

Anatomia degli attacchi alle password

Per capire perché certe best practice funzionano e altre no, è necessario comprendere come vengono realmente attaccate le password. Questo capitolo descrive le tecniche principali a scopo informativo e difensivo: l'obiettivo è riconoscere il rischio, non fornire una guida operativa offensiva.

3.1 Attacco a forza bruta (brute force)

Consiste nel provare sistematicamente tutte le combinazioni possibili di caratteri fino a trovare quella corretta. La sua efficacia dipende esclusivamente dallo spazio delle chiavi (lunghezza e alfabeto utilizzato) e dalla velocità di calcolo dell'attaccante.

INFO

Lo spazio delle chiavi cresce in modo esponenziale con la lunghezza. Una password di 8 caratteri alfanumerici minuscoli ha circa $26^8 \approx 2,1 \times 10^{11}$ combinazioni; una di 12 caratteri ne ha $26^{12} \approx 9,5 \times 10^{16}$. Questo è il motivo tecnico per cui la *lunghezza* è oggi considerata il fattore di sicurezza più importante, più della complessità dei caratteri.

3.2 Attacco a dizionario

Invece di provare tutte le combinazioni, l'attaccante utilizza liste di password note, comuni o trapelate in violazioni precedenti (es. `rockyou.txt`, una delle wordlist più citate nel settore, contenente decine di milioni di password reali esposte in una violazione del 2009). Le varianti includono regole di mutazione (es. sostituire "a" con "@", aggiungere un numero finale) che riproducono le abitudini umane.

3.3 Credential stuffing

È oggi una delle tecniche più efficaci ed economiche. Sfrutta il fatto che la maggior parte degli utenti riutilizza le stesse credenziali su più servizi: combinazioni email/password ottenute da una violazione vengono testate automaticamente su altri siti (banche, e-commerce, social network), spesso tramite botnet distribuite per eludere i rate-limit.

ATTENZIONE

Il credential stuffing non "rompe" nessuna password: sfrutta password corrette ma riutilizzate. È per questo che la mitigazione più efficace non è una password più complessa, ma password *univoche* per ogni servizio, tipicamente gestite tramite un password manager.

3.4 Rainbow table

Una rainbow table è una struttura dati precalcolata che mappa hash a possibili password in chiaro, permettendo di "invertire" rapidamente un hash senza calcolarlo ogni volta. È efficace contro hash

non salati (vedi capitolo 5): con un salt univoco per ogni password, una rainbow table precalcolata diventa inutile, perché bisognerebbe rigenerarla per ogni singolo salt.

3.5 Phishing e ingegneria sociale

Non è un attacco alla password in senso crittografico, ma all'utente. Pagine di login false, email contraffatte, deepfake vocali e messaggi SMS (*smishing*) inducono la vittima a fornire spontaneamente le proprie credenziali. È oggi il vettore di compromissione iniziale più comune nelle violazioni aziendali, ed è anche il motivo principale per cui l'autenticazione a più fattori e le passkey (capitoli 7-8) sono importanti: rendono il furto della sola password insufficiente.

3.6 Keylogging e session hijacking

Malware in grado di registrare la digitazione (keylogger) o di rubare i cookie/token di sessione attivi permettono di compromettere un account anche con password robuste e MFA configurato, se l'attacco avviene *dopo* l'autenticazione. Questo sposta parte della responsabilità dalla password alla sicurezza generale del dispositivo (aggiornamenti, antivirus, evitare software da fonti non verificate).

3.7 Tabella riassuntiva delle contromisure

Attacco	Contromisura principale
Brute force	Lunghezza elevata, rate-limiting, lockout dopo N tentativi
Dizionario	Evitare parole comuni/pattern, controllo contro liste note
Credential stuffing	Password univoche per servizio, MFA, monitoraggio data breach
Rainbow table	Hashing con salt univoco e funzioni costose (bcrypt/Argon2)
Phishing	MFA, passkey, formazione, verifica dominio prima del login
Keylogging/session hijacking	Antivirus, aggiornamenti, timeout di sessione, scadenza token

Fonti

- M. Connors, *The Security Breach that Rocked the World: A Reflection on RockYou* — <https://matt-connors.medium.com/the-security-breach-that-rocked-the-world-a-reflection-on-rockyou-6d636bd84516>
- CISA, *Heightened DDoS Threat Posed by Mirai and Other Botnets* — <https://www.cisa.gov/news-events/alerts/2016/10/14/heightened-ddos-threat-posed-mirai-and-other-botnets>
- OWASP Cheat Sheet Series, *Credential Stuffing Prevention Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html

CAPITOLO 4

Esempi di attacchi famosi alle password

Il capitolo precedente ha descritto le tecniche di attacco in astratto. Questo capitolo le mette in pratica attraverso cinque casi reali, scelti perché ciascuno illustra in modo diverso un fallimento descritto nei capitoli precedenti: storage in chiaro, crittografia debole, hashing senza salt, credenziali di default e riutilizzo di una singola password. Per ciascun caso vengono ricostruiti la storia, le modalità con cui l'attacco è stato condotto e il risultato ottenuto dagli attaccanti.

INFO

Gli episodi descritti sono ricostruiti da fonti pubbliche (articoli giornalistici, analisi di sicurezza, dichiarazioni ufficiali delle aziende coinvolte, atti giudiziari). L'obiettivo è didattico: comprendere le cause profonde di un fallimento di sicurezza, non fornire una guida operativa per replicarlo.

4.1 RockYou (2009): l'archivio che ha cambiato il cracking delle password

Storia. RockYou era un'azienda di San Francisco che sviluppava widget e applicazioni per piattaforme social come Facebook e MySpace, con oltre 100 milioni di utenti attivi mensili nel 2009. Nel dicembre 2009 un attaccante che operava sotto lo pseudonimo "igigi" violò il database degli utenti.

Preparazione e conduzione. L'attacco ha sfruttato una **SQL injection**, una delle vulnerabilità web più elementari e meglio documentate già a quel tempo: un ricercatore di sicurezza della società Imperva aveva individuato e segnalato la falla a RockYou poco prima della violazione, ma l'azienda non l'aveva ancora corretta. L'attaccante ha sfruttato la falla per ottenere accesso diretto al database via query SQL manipulate, senza dover prima rompere alcun hash: le password erano infatti memorizzate **interamente in chiaro**, senza hashing né cifratura di alcun tipo.

ATTENZIONE

RockYou imponeva una lunghezza minima di soli 5 caratteri e non consentiva l'uso di punteggiatura, incentivando di fatto password semplici. Combinato con lo storage in chiaro, questo ha reso l'impatto della violazione massimo a fronte di una vulnerabilità minima.

Risultato. Sono stati esposti gli account di oltre 32 milioni di utenti (la cifra esatta più citata è 32.603.388), comprensivi di email e password in chiaro. L'elenco delle password, filtrato dai duplicati, è diventato la wordlist **rockyou.txt**: con oltre 14 milioni di password reali univoche, è ancora oggi distribuita di default con Kali Linux ed è la prima risorsa utilizzata in qualunque attività di password cracking o di penetration testing, sia da chi attacca sia da chi si difende. RockYou ha patteggiato con la FTC statunitense, impegnandosi ad audit di sicurezza periodici per vent'anni e al pagamento di una sanzione amministrativa.

4.2 Adobe (2013): quando la "crittografia" non è hashing

Storia. Nell'ottobre 2013 Adobe rese noto che un sistema di backup, in attesa di dismissione, era stato compromesso. Le stime iniziali (2,9 milioni di account) sono cresciute rapidamente fino a oltre 152 milioni di record coinvolti.

Preparazione e conduzione. Gli attaccanti hanno avuto accesso al database e ne hanno esfiltrato gli ID utente, le email, le password e — punto critico — i **suggerimenti password (password hint) salvati in chiaro**. Il problema più grave, tuttavia, riguardava il modo in cui le password stesse erano "protette": non con una funzione di hash, ma con **cifratura reversibile Triple DES in modalità ECB**. La modalità ECB cifra blocchi identici di testo in chiaro sempre nello stesso modo: due utenti con la stessa password ottenevano lo stesso testo cifrato, e diventava persino possibile dedurre la lunghezza di una password osservando la struttura del blocco cifrato. Un ricercatore di sicurezza (Jeremi Gosney) dimostrò che, incrociando i pattern ripetuti nei blocchi cifrati con i suggerimenti in chiaro, era possibile risalire alle 100 password più comuni nel giro di circa tre ore, senza nemmeno rompere la cifratura.

Risultato. Quasi 2 milioni di utenti (circa l'1,25% del totale) risultavano avere scelto la stessa identica password, rendendo immediatamente visibile, grazie al difetto della modalità ECB, quali account la condivisero. Diversi grandi servizi (incluso Facebook) utilizzarono il dump per avvisare proattivamente gli utenti che riutilizzavano la stessa combinazione email/password anche sulle proprie piattaforme. Il caso Adobe resta oggi il riferimento didattico standard per spiegare la differenza tra cifratura reversibile e hashing, e per spiegare perché la modalità ECB non va mai usata per dati sensibili.

4.3 LinkedIn (2012, rivelato nel 2016): l'hash senza salt

Storia. Nel giugno 2012 un attaccante pubblicò su un forum russo circa 6,5 milioni di hash di password LinkedIn. L'azienda minimizzò l'accaduto per anni, finché nel maggio 2016 un venditore con lo pseudonimo "Peace" mise in vendita sul dark web un dataset di 167 milioni di credenziali, di cui circa 117 milioni con email e password abbinate, rivelando che la violazione originale era stata molto più estesa di quanto comunicato.

Preparazione e conduzione. Le password erano protette con hashing **SHA-1 senza salt**. SHA-1 era già considerato un algoritmo debole per le password all'epoca (è veloce da calcolare, quindi adatto ad attacchi a forza bruta su larga scala); l'assenza totale di salt ha aggravato il problema, permettendo lo stesso hash per due password identiche e l'uso efficiente di tabelle precalcolate. Il servizio "LeakedSource", che ha analizzato il dataset del 2016, ha dichiarato di aver craccato circa il 90% delle password in appena 72 ore.

Risultato. Oltre alle credenziali compromesse, sono seguite azioni legali contro LinkedIn (risolte con un risarcimento) e l'azienda è migrata a un sistema con salt e hashing più robusto (bcrypt) per tutte le password successive alla violazione. Il caso è spesso citato come l'esempio di riferimento per spiegare perché l'hashing senza salt, anche con un algoritmo non immediatamente "rotto", non è sufficiente a proteggere grandi quantità di credenziali nel tempo.

4.4 Mirai e l'attacco a Dyn (2016): quando il problema sono le credenziali di fabbrica

Storia. Nell'ottobre 2016 un'ondata di attacchi DDoS (*Distributed Denial of Service*) di intensità senza precedenti colpì Dyn, un fornitore di servizi DNS critico per l'infrastruttura di Internet, rendendo temporaneamente irraggiungibili numerosi servizi di primo piano che ne dipendevano.

Preparazione e conduzione. Alla base dell'attacco c'era **Mirai**, un malware che non sfruttava una vulnerabilità software in senso tradizionale, ma un problema di credenziali: milioni di dispositivi IoT (telecamere, router, videoregistratori digitali) venivano venduti con un numero ristretto di combinazioni di nome utente e password di fabbrica, spesso mai cambiate dagli utilizzatori finali. Mirai scansionava continuamente Internet cercando dispositivi con la porta Telnet aperta e tentava l'accesso provando una lista precaricata di circa 60 combinazioni di credenziali predefinite. Ogni dispositivo compromesso diventava parte di una botnet, pronta a generare traffico massivo su comando.

ATTENZIONE

Mirai non ha "rotto" nessuna password nel senso tecnico discusso nei capitoli precedenti: ha semplicemente provato le password che i produttori avevano già impostato, e che nella maggioranza dei casi non erano mai state cambiate. È l'esempio più chiaro di come anche le password più "indovinabili" — quelle di fabbrica — restino, su scala di milioni di dispositivi, un vettore d'attacco enorme.

Risultato. L'attacco a Dyn ha reso inaccessibili per ore servizi di larghissima diffusione che si appoggiavano alla sua infrastruttura DNS. È stato uno dei DDoS più estesi mai registrati fino a quel momento. L'episodio ha portato a una maggiore attenzione normativa e di settore sulla sicurezza "by design" dei dispositivi IoT, incluso l'obbligo, in alcune giurisdizioni successive, di non utilizzare più password di fabbrica identiche e non modificabili.

4.5 Colonial Pipeline (2021): una sola password, nessun MFA

Storia. Il 7 maggio 2021 Colonial Pipeline, che gestisce circa il 45% del trasporto di carburante della costa orientale degli Stati Uniti, ha subito un attacco ransomware da parte del gruppo DarkSide. L'azienda ha interrotto volontariamente l'intera operatività della pipeline per giorni, causando carenze di carburante, acquisti di panico e la dichiarazione dello stato di emergenza da parte del Presidente degli Stati Uniti.

Preparazione e conduzione. A differenza dei casi precedenti, qui non c'è stata alcuna violazione tecnica sofisticata: gli attaccanti sono entrati nella rete aziendale tramite un **account VPN non più in uso ma ancora attivo**, protetto da una password che, secondo le testimonianze rese al Congresso statunitense, era "complessa" ma era stata **riutilizzata** su un altro servizio violato in precedenza, ed era quindi già disponibile tra le credenziali in circolazione sul dark web. Quell'account VPN **non aveva l'autenticazione a più fattori attiva**: una sola password, per quanto robusta nella forma, è stata sufficiente per ottenere un primo punto d'appoggio nella rete. Gli attaccanti sono rimasti nella rete per diversi giorni prima di sferrare l'attacco ransomware, esfiltrando circa 100 GB di dati.

Risultato. Colonial Pipeline ha pagato un riscatto di 75 bitcoin (circa 4,4 milioni di dollari all'epoca); il Dipartimento di Giustizia statunitense è successivamente riuscito a recuperarne una parte. L'incidente è diventato il caso di studio di riferimento per spiegare, a livello dirigenziale e non solo tecnico, perché il riutilizzo delle password e l'assenza di MFA anche su un singolo account ritenuto "secondario" possano avere conseguenze su un'infrastruttura nazionale critica.

4.6 Cosa hanno in comune questi cinque casi

Caso	Causa profonda (collegata ai capitoli precedenti)
RockYou (2009)	Nessun hashing: password salvate in chiaro (cap. 5)
Adobe (2013)	Cifratura reversibile (ECB) al posto di hashing (cap. 5)
LinkedIn (2012/2016)	Hashing senza salt (cap. 5)
Mirai / Dyn (2016)	Credenziali di default mai cambiate (cap. 3, cap. 10)
Colonial Pipeline (2021)	Riutilizzo password + assenza di MFA (cap. 3, cap. 8)

Nessuno di questi casi è stato causato da un attacco crittografico sofisticato: in ognuno, la causa profonda è esattamente uno degli errori discussi nei capitoli precedenti di questo documento, applicato però su scala milionaria o su un'infrastruttura critica.

Fonti

- TechCrunch, *RockYou Hack: From Bad To Worse* (dic. 2009) — <https://techcrunch.com/2009/12/14/rockyou-hack-security-myspace-facebook-passwords/>
- Computerworld, *RockYou settles FTC charges related to 2009 breach* — <https://www.computerworld.com/article/1527859/rockyou-settles-ftc-charges-related-to-2009-breach.html>
- GeekWire / Sophos Naked Security, *Adobe breach hit more than 150 million usernames and passwords* (nov. 2013) — <https://www.geekwire.com/2013/report-adobe-breach-hit-150-million-usernames-passwords/>
- B. Schneier, *Cryptographic Blunders Revealed by Adobe's Password Leak*, Schneier on Security (nov. 2013) — https://www.schneier.com/blog/archives/2013/11/cryptographic_b.html
- Wikipedia, *2012 LinkedIn hack* — https://en.wikipedia.org/wiki/2012_LinkedIn_hack
- TechCrunch, *117 million LinkedIn emails and passwords from a 2012 hack just got posted online* (mag. 2016) — <https://techcrunch.com/2016/05/18/117-million-linkedin-emails-and-passwords-from-a-2012-hack-just-got-posted-online/>
- CISA, *Heightened DDoS Threat Posed by Mirai and Other Botnets* — <https://www.cisa.gov/news-events/alerts/2016/10/14/heightened-ddos-threat-posed-mirai-and-other-botnets>
- Wikipedia, *Colonial Pipeline ransomware attack* — https://en.wikipedia.org/wiki/Colonial_Pipeline_ransomware_attack
- Sophos, *What IT security teams can learn from the Colonial Pipeline ransomware attack* (giu. 2021) — <https://www.sophos.com/en-us/blog/what-it-security-teams-can-learn-from-the-colonial-pipeline-ransomware-attack>

CAPITOLO 5

Crittografia e hashing delle password

5.1 Hash vs cifratura: una distinzione fondamentale

Un errore concettuale comune è confondere **cifratura** (reversibile, con una chiave) e **hashing** (unidirezionale, non reversibile). Le password non dovrebbero mai essere cifrate in modo reversibile: dovrebbero essere sottoposte a una funzione di hash, che produce un'impronta a lunghezza fissa dalla quale, idealmente, non è possibile risalire all'input originale.

INFO

Se un servizio è in grado di *mostrarti* la tua vecchia password (anche solo via email "password dimenticata: era X"), significa che la stava conservando in modo non sicuro. Un sistema progettato correttamente può solo permetterti di *reimpostarla*, mai di recuperarla.

5.2 L'evoluzione degli algoritmi di hashing

5.2.1 MD5 e SHA-1: veloci ma non più adeguati

MD5 (1992) e SHA-1 (1995) sono funzioni di hash crittografico progettate per essere *veloci*, una caratteristica desiderabile per l'integrità dei dati ma **indesiderabile** per le password: la velocità di calcolo è esattamente ciò che permette a un attaccante di provare miliardi di combinazioni al secondo su hardware moderno (in particolare GPU).

ATTENZIONE

MD5 e SHA-1 (anche con salt) sono oggi considerati inadeguati per l'hashing di password. Restano validi per altri usi (checksum di integrità), ma non devono essere utilizzati per proteggere credenziali.

5.2.2 Salting: contro le rainbow table

Il **salt** è un valore casuale univoco, generato per ogni password e concatenato a essa prima dell'hashing. Garantisce che due utenti con la stessa password ottengano hash completamente diversi, rendendo inutili le rainbow table precalcolate.

Listing 5.1: Esempio concettuale di hashing con salt (pseudo-codice)

```
import os, hashlib

def hash_password(password: str) -> tuple[bytes, bytes]:
    salt = os.urandom(16)                # salt casuale a 16 byte
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), salt, 200_000
    )
    return salt, digest                  # entrambi vanno salvati a DB
```

5.2.3 Key stretching: bcrypt, scrypt, Argon2

La risposta moderna al problema della velocità è il **key stretching**: algoritmi progettati per essere deliberatamente lenti e costosi dal punto di vista computazionale, rendendo gli attacchi a forza bruta su larga scala impraticabili anche con hardware specializzato.

Algoritmo	Anno	Caratteristiche principali
PBKDF2	2000	Iterazioni configurabili, ampiamente supportato, ma vulnerabile ad attacchi GPU/ASIC su larga scala se le iterazioni sono basse
bcrypt	1999	Basato su Blowfish, "cost factor" configurabile, standard de facto per anni
scrypt	2009	Oltre al costo CPU, richiede molta memoria (memory-hard), penalizzando hardware dedicato (ASIC/FPGA)
Argon2	2015	Vincitore della Password Hashing Competition; varianti Argon2i, Argon2d, Argon2id (consigliata)

BEST PRACTICE

Per nuovi sviluppi, lo standard consigliato oggi è **Argon2id**, con parametri di memoria, iterazioni e parallelismo calibrati sull'hardware del server. Dove non disponibile, **bcrypt** resta una scelta solida e ben supportata.

5.3 Peppering: un livello opzionale aggiuntivo

Alcuni sistemi aggiungono un **pepper**: un valore segreto, condiviso da tutte le password e conservato separatamente dal database (es. in un secret manager o HSM), che si aggiunge al salt. A differenza del salt, il pepper non viene salvato insieme all'hash: questo significa che, anche in caso di esfiltrazione completa del database, un attaccante senza accesso al pepper non può iniziare un attacco offline.

5.4 Cosa significa, in pratica, per chi sviluppa software

- Non implementare mai un proprio schema di hashing "fatto in casa": usare librerie mature e mantenute (es. **bcrypt**, **argon2-cffi** in Python, **libsodium**).
- Non limitare artificialmente la lunghezza massima della password lato server (errore comune che, paradossalmente, riduce la sicurezza).
- Confrontare gli hash con funzioni a tempo costante (*constant-time comparison*) per evitare timing attack.
- Pianificare la **migrazione degli algoritmi**: un sistema ben progettato deve poter ri-hashare le password con un algoritmo più moderno al successivo login, senza richiedere un reset forzato a tutti gli utenti.

Fonti

- OWASP Cheat Sheet Series, *Password Storage Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Cheat Sheet Series, *Cryptographic Storage Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- Password Hashing Competition, repository ufficiale di Argon2 — <https://github.com/P-H-C/p>

hc-winner-argon2

- B. Schneier, *Cryptographic Blunders Revealed by Adobe's Password Leak*, Schneier on Security — https://www.schneier.com/blog/archives/2013/11/cryptographic_b.html

CAPITOLO 6

Le linee guida NIST e il cambio di paradigma

6.1 Dal NIST SP 800-63 (2003) al NIST SP 800-63B (2017+)

Il documento di riferimento più citato a livello internazionale in materia di autenticazione è il *NIST Special Publication 800-63*, pubblicato dal National Institute of Standards and Technology statunitense. La revisione del 2017 (800-63B), aggiornata negli anni successivi, ha segnato un cambio di paradigma rispetto alle regole "classiche" degli anni 2000.

6.2 Cosa cambia rispetto alle policy tradizionali

Approccio tradizionale (2003)	Approccio NIST attuale (2017+)
Lunghezza minima breve (8 caratteri)	Lunghezza minima maggiore (almeno 8, idealmente 12+), senza limite massimo eccessivamente basso (consigliato fino a 64)
Obbligo di maiuscole, numeri, simboli	Nessun requisito di composizione obbligatorio: l'utente può scegliere liberamente il formato
Scadenza forzata periodica (30/60/90 gg)	Nessuna scadenza forzata, salvo evidenza di compromissione
Suggerimenti (<i>password hint</i>)	Suggerimenti vietati: spesso facilitano l'attacco più che aiutare l'utente
Nessun controllo su password comuni	Obbligo di verificare la password contro liste di credenziali note come compromesse

INFO

Le linee guida NIST si applicano formalmente alle agenzie federali statunitensi, ma sono diventate un punto di riferimento de facto adottato (in tutto o in parte) da gran parte dell'industria tecnologica mondiale, incluse le policy di autenticazione di numerosi grandi provider cloud e servizi consumer.

6.3 Perché la rotazione forzata è stata abbandonata

La ricerca accademica (in particolare gli studi citati dallo stesso NIST e da Microsoft Research) ha mostrato che la scadenza forzata periodica produce un effetto controproducente: costretti a cambiare password regolarmente, gli utenti tendono a generare varianti minime e predicibili della password precedente (es. incrementando un numero finale), oppure ad annotarle in luoghi insicuri (foglietti, file di testo non protetti). Il costo cognitivo della rotazione supera quindi il beneficio di sicurezza, a meno che non vi sia evidenza concreta di una compromissione.

6.4 Perché la complessità obbligatoria non funziona

Imporre maiuscole, numeri e simboli non aumenta in modo significativo l'entropia reale delle password scelte dagli utenti, perché la distribuzione delle sostituzioni è altamente predicibile

(es. "S" → "\$", "o" → "0", "a" → "@", aggiunta di "1" o "!" alla fine). Un attaccante che conosce questi pattern li incorpora semplicemente nelle proprie wordlist, vanificando gran parte del beneficio teorico.

6.5 Il principio guida: lunghezza > complessità

BEST PRACTICE

Una passphrase lunga e facile da ricordare, come `cavalloblubicicletta42` (24 caratteri), è matematicamente più resistente a un attacco a forza bruta rispetto a una password breve e complessa come `P@ss1!` (6 caratteri), pur essendo più semplice da memorizzare per un essere umano.

6.6 Cosa NIST raccomanda concretamente (sintesi)

- Lunghezza minima di almeno 8 caratteri per utenti, 6 per password generate automaticamente dal sistema; lunghezza massima consentita fino ad almeno 64 caratteri.
- Accettare tutti i caratteri stampabili, inclusi gli spazi, per permettere passphrase naturali.
- Verificare la password scelta contro dizionari di password comuni, parole di senso comune e credenziali note da violazioni precedenti, rifiutandola se presente.
- Eliminare i requisiti di composizione obbligatoria (mix di maiuscole/minuscole/numeri/simboli).
- Eliminare la scadenza periodica forzata, salvo sospetta compromissione.
- Implementare limiti ai tentativi di accesso (rate limiting / lockout temporaneo) per contrastare la forza bruta online.
- Offrire e incentivare l'autenticazione a più fattori, specialmente per account sensibili.

Fonti

- NIST, *SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management* — <https://pages.nist.gov/800-63-3/sp800-63b.html>
- NIST, *SP 800-63-3: Digital Identity Guidelines* (volume cappello della suite) — <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63-3.pdf>
- R. McMillan, *The Man Who Wrote Those Password Rules Has a New Tip*, The Wall Street Journal (ago. 2017) — riportato in BizTech Magazine, <https://biztechmagazine.com/article/2017/08/password-rule-guru-bill-burr-now-regrets-his-advice>

CAPITOLO 7

Password manager

7.1 Il problema che risolvono

Seguire alla lettera le best practice del capitolo 6 — una password lunga, univoca, non riutilizzata, per *ogni* servizio — è praticamente impossibile da gestire a memoria quando il numero di account supera la decina. I password manager risolvono questo problema spostando il carico cognitivo dall'utente al software: l'utente deve ricordare una sola password (quella "master"), mentre il gestore genera, memorizza e compila automaticamente tutte le altre.

7.2 Come funzionano: il modello di sicurezza

Un password manager moderno e ben progettato si basa su un'architettura **zero-knowledge**: i dati vengono cifrati e decifrati esclusivamente sul dispositivo dell'utente (*client-side*), utilizzando una chiave derivata dalla master password tramite una funzione di key derivation lenta (es. PBKDF2, Argon2). Il fornitore del servizio, anche se gestisce la sincronizzazione cloud, non è mai in possesso della master password né, in teoria, della chiave di decifratura.

INFO

"Zero-knowledge" non significa che il fornitore non possa subire una violazione: significa che, anche in caso di violazione del suo database, l'attaccante otterrebbe solo dati cifrati, inutilizzabili senza la master password dell'utente.

7.3 Tipologie di password manager

Tipologia	Caratteristiche
Cloud-based (es. servizi commerciali con sync multi-dispositivo)	Comodi, sincronizzazione automatica tra dispositivi; il modello di sicurezza dipende dall'implementazione zero-knowledge del fornitore
Locali / offline (es. database cifrati locali)	Nessuna dipendenza da terzi, massimo controllo; sincronizzazione manuale o tramite storage personale (es. proprio cloud privato)
Integrati nel browser	Comodi e gratuiti, ma generalmente con funzionalità di sicurezza più limitate rispetto a soluzioni dedicate
Hardware (chiavi di sicurezza con elemento sicuro)	Massima resistenza a malware ed estrazione remota della chiave, usati spesso in combinazione con MFA/passkey

7.4 La master password: l'unico punto debole rimasto

Poiché tutta la sicurezza del sistema si concentra su un'unica credenziale, la master password deve essere trattata con un livello di attenzione superiore a qualsiasi altra password:

BEST PRACTICE

- Deve essere una passphrase lunga (20+ caratteri), unica, non riutilizzata altrove.
- Non deve mai essere salvata in chiaro, né condivisa.
- L'accesso al password manager dovrebbe essere protetto da MFA, quando supportato.
- È consigliabile annotare una copia di backup della master password (non delle singole password) in un luogo fisico sicuro, per evitare il rischio di lockout permanente.

7.5 Obiezioni comuni e risposta tecnica

“Se il password manager viene violato, perdo tutto.”

È un rischio reale, ma va confrontato con l'alternativa: riutilizzare poche password su molti siti significa che la violazione di un singolo servizio compromette automaticamente tutti gli altri. Un'architettura zero-knowledge ben implementata mitiga fortemente questo rischio.

“Preferisco ricordarle a memoria.”

È sostenibile solo per un numero molto ridotto di account critici. Per tutto il resto, la memoria umana non scala e porta inevitabilmente al riutilizzo.

“Uso il browser, mi basta.”

È una soluzione migliore di niente, ma generalmente offre meno funzionalità di sicurezza avanzate (es. controllo data breach, condivisione sicura, audit delle password debili) rispetto a soluzioni dedicate.

7.6 Criteri pratici per scegliere un password manager

1. Architettura zero-knowledge documentata e, idealmente, sottoposta ad audit di sicurezza indipendenti pubblicati.
2. Supporto a MFA per l'accesso al vault.
3. Possibilità di esportare i dati in formato standard, per evitare il vendor lock-in.
4. Generatore di password integrato, configurabile per lunghezza e tipologia di caratteri.
5. Funzione di controllo delle password compromesse (data breach monitoring).
6. Supporto multiplatforma adeguato al proprio contesto d'uso (desktop, mobile, browser).

Fonti

- OWASP Cheat Sheet Series, *Password Storage Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- NIST, *SP 800-63B: Digital Identity Guidelines* — <https://pages.nist.gov/800-63-3/sp800-63b.html>

CAPITOLO 8

Autenticazione a più fattori (MFA/2FA)

8.1 Il principio: qualcosa che sai, hai, sei

L'autenticazione a più fattori (*Multi-Factor Authentication*, MFA) si basa sulla combinazione di almeno due categorie indipendenti di evidenza:

Fattore	Esempi
Qualcosa che sai (<i>knowledge</i>)	Password, PIN, risposta a domanda di sicurezza
Qualcosa che hai (<i>possession</i>)	Smartphone, token hardware, smart card, chiave di sicurezza FIDO2
Qualcosa che sei (<i>inherence</i>)	Impronta digitale, riconoscimento facciale, biometria comportamentale

INFO

Per essere considerata "a più fattori" e non semplicemente "a più passaggi", l'autenticazione deve combinare categorie *diverse*. Inserire una password e poi una domanda di sicurezza (entrambe nella categoria "qualcosa che sai") non costituisce un vero secondo fattore.

8.2 Tipologie di secondo fattore, dalla meno alla più sicura

8.2.1 SMS OTP (codice via SMS)

Storicamente il metodo più diffuso, oggi considerato il **meno sicuro** tra le opzioni MFA comuni. È vulnerabile a *SIM swapping* (l'attaccante convince l'operatore telefonico a trasferire il numero su una propria SIM) e all'intercettazione tramite vulnerabilità del protocollo SS7. Meglio di niente, ma da evitare quando esistono alternative.

8.2.2 App di autenticazione TOTP (Time-based One-Time Password)

Generano un codice numerico che cambia ogni 30 secondi, basato su un segreto condiviso al momento della configurazione e sull'orario corrente (RFC 6238). Non dipendono dalla rete telefonica e sono significativamente più resistenti al SIM swapping rispetto agli SMS.

Listing 8.1: Principio di funzionamento di TOTP (semplificato)

```
codice = HMAC-SHA1(segreto_condiviso, timestamp_corrente // 30s)
```

8.2.3 Notifiche push

L'app del provider invia una notifica ("Stai cercando di accedere da Milano, confermi?") che l'utente approva o rifiuta. Comode, ma vulnerabili al *MFA fatigue* (o *push bombing*): l'attaccante invia ripetute richieste di approvazione finché la vittima, per stanchezza o distrazione, ne approva una per errore.

ATTENZIONE

Diverse violazioni di alto profilo degli ultimi anni sono state attribuite proprio a MFA fatigue. Per mitigarlo, molti provider hanno introdotto la verifica di un codice numerico visualizzato sullo schermo di login, da inserire nella notifica push, rendendo l'approvazione "alla cieca" più difficile.

8.2.4 Chiavi di sicurezza hardware (FIDO2/U2F)

Dispositivi fisici (USB, NFC, Bluetooth) che utilizzano crittografia a chiave pubblica per autenticarsi: la chiave privata non lascia mai il dispositivo. Sono resistenti al phishing per costruzione, perché il protocollo verifica anche l'identità del dominio richiedente: una chiave di sicurezza non risponderà mai a una pagina di login falsa, anche se visivamente identica a quella originale.

BEST PRACTICE

Per account ad alto valore (email principale, password manager, accesso amministrativo, criptovalute) l'uso di una chiave di sicurezza hardware FIDO2 è oggi considerato il livello di protezione più alto disponibile per il pubblico generale.

8.3 Tabella comparativa

Metodo	Sicurezza	Comodità	Principale vulnerabilità
SMS OTP	Bassa	Alta	SIM swapping, intercettazione SS7
TOTP (app)	Media-Alta	Media	Furto del segreto se il dispositivo è compromesso
Notifica push	Media	Alta	MFA fatigue / push bombing
Chiave hardware FIDO2	Molto alta	Media	Smarrimento fisico (mitigato con chiave di backup)

8.4 Raccomandazione pratica

Qualsiasi forma di MFA è meglio di nessuna. Dove possibile, è preferibile evitare gli SMS come unico fattore e preferire app TOTP o, per gli account più critici, chiavi di sicurezza hardware, mantenendo sempre almeno un metodo di recupero di backup configurato in anticipo.

Fonti

- OWASP Cheat Sheet Series, *Multifactor Authentication Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html
- M'Raihi et al., *RFC 6238: TOTP — Time-Based One-Time Password Algorithm*, IETF — <https://datatracker.ietf.org/doc/html/rfc6238>
- Sophos, *What IT security teams can learn from the Colonial Pipeline ransomware attack* — <https://www.sophos.com/en-us/blog/what-it-security-teams-can-learn-from-the-colonial-pipeline-ransomware-attack>

CAPITOLO 9

Passwordless e passkey

9.1 Perché eliminare del tutto la password

Ogni capitolo precedente ha descritto, in fondo, come "compensare" i limiti intrinseci della password: hashing per proteggerla a riposo, MFA per proteggerla in transito, password manager per gestirne la complessità. L'approccio **passwordless** affronta il problema in modo radicale: eliminare la password come segreto condiviso da trasmettere, sostituendola con la crittografia a chiave pubblica.

9.2 FIDO2 e WebAuthn: le basi tecniche

FIDO2 è uno standard aperto (FIDO Alliance, in collaborazione con il W3C) che comprende il protocollo **WebAuthn**, integrato nativamente nei principali browser e sistemi operativi moderni. Il principio di funzionamento:

1. Al momento della registrazione, il dispositivo dell'utente genera una coppia di chiavi crittografiche (pubblica/privata) univoca per quel servizio.
2. La chiave **privata** non lascia mai il dispositivo (è protetta da un elemento sicuro hardware, da biometria locale o da un PIN locale).
3. La chiave **pubblica** viene inviata e memorizzata dal servizio.
4. Ad ogni login, il servizio invia una "sfida" (*challenge*) casuale; il dispositivo la firma con la chiave privata e restituisce la firma, verificabile dal servizio con la chiave pubblica.

INFO

Non esiste alcun segreto condiviso da trasmettere in rete o da memorizzare lato server. Un database compromesso conterrebbe solo chiavi pubbliche, intrinsecamente inutili per un attaccante: non permettono di autenticarsi da nessuna parte.

9.3 Le passkey: WebAuthn reso utilizzabile da tutti

Le **passkey** sono, in sostanza, credenziali WebAuthn pensate per l'utente comune, con un'aggiunta cruciale: la possibilità di sincronizzazione cifrata tra i propri dispositivi tramite il portachiavi cloud del proprio ecosistema (es. gestori di credenziali integrati nei sistemi operativi e nei principali password manager), risolvendo il problema storico di FIDO2 ("cosa succede se perdo il dispositivo?").

9.4 Perché le passkey sono resistenti al phishing

A differenza di una password, che un utente può digitare (per errore) su qualsiasi pagina, anche falsa, una passkey è **legata crittograficamente al dominio** per cui è stata creata. Il browser/-sistema operativo verifica automaticamente che il dominio richiedente corrisponda a quello di reg-

istrazione: una pagina di phishing, anche se identica visivamente all'originale, non può richiedere con successo l'uso della passkey.

BEST PRACTICE

Questo è il vantaggio più significativo delle passkey rispetto a qualsiasi combinazione password+MFA tradizionale (eccetto le chiavi hardware FIDO2, che condividono lo stesso principio): l'elemento umano, storicamente l'anello più debole, viene rimosso dall'equazione del phishing.

9.5 Limiti e considerazioni pratiche attuali

- Non tutti i servizi supportano ancora le passkey come metodo di autenticazione primario; l'adozione è in forte crescita ma non ancora universale.
- La sincronizzazione cross-ecosistema (es. tra fornitori diversi) può risultare meno fluida rispetto alla sincronizzazione all'interno dello stesso ecosistema.
- È comunque buona norma mantenere un metodo di recupero alternativo configurato, in caso di smarrimento di tutti i dispositivi sincronizzati.

9.6 Stato di transizione: convivenza con le password

Nella pratica attuale, la maggior parte dei servizi offre le passkey come *opzione aggiuntiva* rispetto alla password tradizionale, non come sostituzione obbligatoria. La strategia più realistica nel medio termine è quindi ibrida: adottare passkey dove disponibili, e applicare rigorosamente le best practice dei capitoli precedenti (password manager, MFA) per tutto ciò che ancora richiede una password.

Fonti

- W3C, *Web Authentication: An API for accessing Public Key Credentials (WebAuthn) Level 2* — <https://www.w3.org/TR/webauthn-2/>
- FIDO Alliance, *FIDO2: WebAuthn & CTAP* — <https://fidoalliance.org/fido2/>
- NIST, *SP 800-63B: Digital Identity Guidelines* (incl. supplemento sulle passkey) — <https://pages.nist.gov/800-63-3/sp800-63b.html>

CAPITOLO 10

Errori comuni da evitare

10.1 Lato utente

Riutilizzare la stessa password su più servizi

È l'errore singolo più impattante: trasforma la violazione di un servizio minore nella compromissione di tutti gli account collegati alla stessa combinazione email/password.

Pattern predicibili e sostituzioni "ovvie"

Sostituzioni come "a"→"@" o l'aggiunta di un numero finale incrementale sono già incluse nelle wordlist degli attaccanti: offrono un falso senso di sicurezza.

Informazioni personali nella password

Nomi, date di nascita, nomi di animali domestici: sono spesso pubblicamente reperibili (social network) e tra i primi tentativi in un attacco mirato.

Condividere le password via chat o email

Anche con persone di fiducia, questi canali non sono progettati per la gestione sicura di segreti e lasciano tracce difficili da eliminare completamente.

Ignorare le notifiche di data breach

Quando un servizio segnala una violazione, la password compromessa va cambiata immediatamente, ovunque sia stata riutilizzata.

Domande di sicurezza con risposte verificabili

"Nome da nubile della madre" o "città di nascita" sono spesso ricostruibili tramite ricerche pubbliche o ingegneria sociale.

10.2 Lato sviluppatore / amministratore di sistema

Limitare artificialmente la lunghezza massima

Bloccare le password a 8-10 caratteri impedisce all'utente di usare passphrase lunghe e sicure, vanificando le best practice del capitolo 6.

Hashing debole o assente

Usare MD5/SHA-1 senza salt, o peggio salvare le password in chiaro, resta sorprendentemente comune in sistemi legacy o mal progettati.

Messaggi di errore troppo specifici

Un messaggio "email non trovata" vs "password errata" permette a un attaccante di enumerare quali email sono registrate sul sistema (*user enumeration*).

Assenza di rate-limiting sul login

Senza un limite ai tentativi falliti, un endpoint di login diventa un bersaglio aperto per attacchi a forza bruta automatizzati.

Invio della password in chiaro via email

Sia in fase di registrazione che di recupero: la password non dovrebbe mai transitare in chiaro su un canale non progettato per segreti.

Reset password con token non scadenti

I link di reset devono avere una validità limitata nel tempo e devono essere invalidati dopo il primo utilizzo.

Log che registrano le password

Anche per errore di debug, le password (anche se poi hashate) non devono mai apparire nei log applicativi.

ATTENZIONE

Un sistema che impone requisiti di complessità eccessivi ma non implementa hashing adeguato offre una falsa sensazione di sicurezza: la robustezza percepita dall'utente non corrisponde alla robustezza reale del sistema che la conserva.

10.3 Errori di percezione comuni

- Credere che una password "complessa" sia automaticamente sicura, indipendentemente dalla lunghezza.
- Credere che il proprio account non sia "interessante" per un attaccante: nella maggior parte dei casi gli attacchi sono automatizzati e non mirati a una persona specifica, ma a un volume di credenziali.
- Credere che l'MFA renda una password debole "accettabile": l'MFA è un livello aggiuntivo, non un sostituto di buone credenziali.
- Sottovalutare il rischio dei dispositivi condivisi o pubblici per l'accesso ad account personali.

Fonti

- OWASP Cheat Sheet Series, *Authentication Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Have I Been Pwned, *Adobe Data Breach* — <https://haveibeenpwned.com/breach/Adobe>

CAPITOLO 11

Checklist operativa di sintesi

Questo capitolo riassume in forma operativa quanto descritto nei capitoli precedenti, separando la prospettiva dell'utente finale da quella di chi progetta o amministra sistemi che gestiscono autenticazione.

11.1 Per l'utente

1. Usa un password manager con architettura zero-knowledge per generare e memorizzare password univoche per ogni servizio.
2. Proteggi la master password del password manager come la credenziale più importante che possiedi: lunga, unica, mai riutilizzata, protetta da MFA.
3. Preferisci passphrase lunghe (16+ caratteri) a password brevi e complesse, dove devi memorizzarne una a mente.
4. Attiva l'MFA su tutti gli account che lo supportano, preferendo app TOTP o chiavi hardware agli SMS.
5. Attiva le passkey dove disponibili, specialmente per gli account più critici.
6. Non riutilizzare mai la stessa password su più servizi, in nessun caso.
7. Verifica periodicamente se le tue credenziali sono apparse in violazioni note, e cambia immediatamente le password coinvolte.
8. Non condividere password via chat, email o messaggistica non cifrata.
9. Configura sempre un metodo di recupero alternativo per account critici ed MFA, prima che serva.

11.2 Per chi sviluppa o amministra sistemi

1. Usa Argon2id (o bcrypt come alternativa solida) per l'hashing, con salt univoco per ogni password.
2. Non limitare la lunghezza massima della password a valori bassi; consenti almeno 64 caratteri.
3. Non imporre requisiti di composizione obbligatoria (mix di simboli/maiuscole/numeri); verifica invece contro liste di password comuni/compromesse.
4. Elimina la scadenza forzata periodica, salvo evidenza concreta di compromissione.
5. Implementa rate-limiting e logout temporaneo sui tentativi di login.
6. Offri MFA, idealmente con supporto a TOTP e/o FIDO2/WebAuthn, non solo SMS.
7. Valuta l'adozione di passkey come opzione di autenticazione, in aggiunta alla password.
8. Non inviare mai password in chiaro via email; usa token di reset con scadenza breve e a

uso singolo.

9. Assicuratevi che nessun log applicativo registri password, nemmeno in fase di debug.
10. Pianifica una strategia di migrazione degli algoritmi di hashing nel tempo, senza richiedere reset forzati massivi.

INFO

La sicurezza dell'autenticazione non è un singolo controllo, ma la somma di più livelli indipendenti: hashing robusto, policy ragionevoli, MFA, e — quando possibile — eliminazione della password stessa tramite passkey. Nessun singolo livello, da solo, è sufficiente.

Fonti

- NIST, *SP 800-63B: Digital Identity Guidelines* — <https://pages.nist.gov/800-63-3/sp800-63b.html>
- OWASP Cheat Sheet Series, *Password Storage Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Cheat Sheet Series, *Multifactor Authentication Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html

Glossario

Argon2 / Argon2id

Funzione di key derivation vincitrice della Password Hashing Competition (2015), considerata oggi lo standard più robusto per l'hashing delle password; la variante **id** combina resistenza ad attacchi side-channel e ad attacchi con hardware dedicato.

bcrypt

Algoritmo di hashing per password basato sul cifrario Blowfish (1999), con un "cost factor" configurabile per regolarne il costo computazionale nel tempo.

Brute force

Attacco che consiste nel provare sistematicamente tutte le combinazioni possibili di caratteri fino a individuare quella corretta.

Credential stuffing

Attacco automatizzato che utilizza combinazioni email/password ottenute da violazioni precedenti per tentare l'accesso ad altri servizi, sfruttando il riutilizzo delle credenziali.

FIDO2

Standard aperto per l'autenticazione basata su crittografia a chiave pubblica, sviluppato dalla FIDO Alliance, alla base di chiavi di sicurezza hardware e passkey.

Hashing

Trasformazione unidirezionale di un dato (es. una password) in un'impronta a lunghezza fissa, dalla quale non è possibile risalire all'input originale.

Key stretching

Tecnica che rende deliberatamente lento e costoso il calcolo di una funzione di hash, per contrastare attacchi a forza bruta su larga scala (es. bcrypt, scrypt, Argon2).

MFA / 2FA

Multi-Factor Authentication / Two-Factor Authentication: autenticazione che richiede la combinazione di almeno due fattori indipendenti (qualcosa che sai, hai, sei).

MFA fatigue (push bombing)

Tecnica con cui un attaccante invia ripetute richieste di approvazione MFA via notifica push, nella speranza che la vittima ne approvi una per errore o stanchezza.

NIST SP 800-63B

Pubblicazione del National Institute of Standards and Technology statunitense contenente le linee guida moderne in materia di autenticazione digitale, che hanno superato le policy tradizionali di complessità e rotazione forzata.

Passkey

Credenziale basata su WebAuthn/FIDO2 pensata per l'utente comune, sincronizzabile in modo cifrato tra i propri dispositivi, che elimina la necessità di una password tradizionale.

Passphrase

Password composta da una sequenza di parole o da una frase, generalmente più lunga di una password tradizionale ma più facile da memorizzare, e quindi spesso più sicura a parità di sforzo cognitivo.

Pepper

Valore segreto condiviso, conservato separatamente dal database delle password (es. in un secret manager), usato come ulteriore livello di protezione oltre al salt.

Phishing

Tecnica di ingegneria sociale che induce la vittima a fornire spontaneamente credenziali o informazioni sensibili, tipicamente tramite pagine di login o comunicazioni contraffatte.

Rainbow table

Struttura dati precalcolata che mappa hash a possibili valori in chiaro, utilizzata per accelerare l'inversione di hash non protetti da salt.

Salt

Valore casuale univoco generato per ogni password e combinato con essa prima dell'hashing, per impedire l'uso di rainbow table precalcolate e garantire hash diversi anche per password identiche.

Session hijacking

Furto o riutilizzo non autorizzato di un token/cookie di sessione attivo, che permette di impersonare un utente già autenticato senza conoscere la sua password.

SIM swapping

Tecnica con cui un attaccante convince un operatore telefonico a trasferire il numero della vittima su una SIM in proprio possesso, per intercettare codici OTP via SMS.

TOTP

Time-based One-Time Password: codice numerico temporaneo generato da un'app di autenticazione, basato su un segreto condiviso e sull'orario corrente (RFC 6238).

WebAuthn

Standard del W3C, parte di FIDO2, che definisce l'API tramite cui i browser e i siti web implementano l'autenticazione basata su crittografia a chiave pubblica.

Zero-knowledge (architettura)

Modello in cui il fornitore di un servizio (es. un password manager) non ha mai accesso ai dati in chiaro dell'utente, perché la cifratura/decifratura avviene esclusivamente sul dispositivo dell'utente stesso.

Cheat Sheet — Password in pratica

Una pagina di sintesi pratica, pensata per essere consultata rapidamente o stampata separatamente.

1. Regole d'oro

- Lunghezza > complessità: minimo 12-16 caratteri, idealmente una passphrase.
- Una password **univoca** per ogni servizio. Sempre.
- Mai informazioni personali (nomi, date, animali).
- Mai la stessa password riusata "con una variante".
- Niente scadenza forzata senza motivo: meglio cambiarla solo se compromessa.

2. Strumenti da usare

- Password manager con architettura zero-knowledge.
- MFA ovunque disponibile: TOTP o chiave hardware > SMS.
- Passkey (FIDO2/WebAuthn) dove offerte dal servizio.
- Servizio di verifica data breach per le proprie email/-credenziali.

3. Cosa NON fare mai

- Riutilizzare la password su più siti.
- Inviare password via email o chat in chiaro.
- Usare domande di sicurezza con risposte pubbliche/verificabili.
- Salvare password in file di testo non cifrati.
- Approvare notifiche MFA non richieste personalmente.

4. Per chi sviluppa

- Hashing: **Argon2id** (o bcrypt), sempre con salt univoco.
- Lunghezza massima consentita: almeno 64 caratteri.
- Nessun requisito di composizione obbligatorio.
- Rate-limiting / lockout sui tentativi di login.
- Reset password: token con scadenza breve, uso singolo.
- MFA: preferire TOTP/FIDO2 a SMS; mai loggare password in chiaro.

5. Livelli di hashing (dal peggiore al migliore)

1. Testo in chiaro — **da evitare sempre**
2. MD5 / SHA-1 senza salt — inadeguato
3. SHA-256 con salt, poche iterazioni — insufficiente
4. PBKDF2 / bcrypt con parametri adeguati — accettabile
5. **Argon2id** con parametri calibrati — consigliato

6. Sicurezza del fattore MFA

SMS OTP	Bassa
App TOTP	Alta
Notifica push	Media
Chiave hardware FIDO2 / Passkey	Molto alta

Documento: password — Creator: 0THack1A — Aggiornamento: Giugno 2026